



The Unified GPU-Native
Multichain Engine:
Measured Benchmarks
of EVM Execution, DEX
Matching, and FHE
from Apple M1 Max
Metal to NVIDIA
GB10 and AMD RDNA

2026

Abstract

We present measured benchmarks of GPU-accelerated Ethereum Virtual Machine (EVM) execution on Apple M1 Max with 32-core Metal GPU and 64 GB unified memory. The GPU EVM kernel achieves 85.9 M opcodes/sec versus 17.7 M opcodes/sec on CPU—a **4.85× peak speedup**. GPU acceleration becomes advantageous above ~500 opcodes per transaction, reaching 2.08× at 2,000 ops/tx, 2.83× at 5,000 ops/tx, and 3.09× at 10,000 ops/tx.

Independent of GPU work, the C++ evmone interpreter with a full state layer achieves 20.9 Ggas/s (best: 23.4 Ggas/s)—**5.1× faster** than the Go EVM’s 4.2–5.4 Ggas/s on EVM-only execution. Profiling confirms that **87% of Go block processing** is secp256k1 ecrecover; parallelizing ecrecover across 10 CPU cores yields 7.1× speedup.

The system is validated by 1,183 tests across C++/Rust/Go, 35 Metal shaders and 11 WGSL shaders (all compiling clean), 14 host dispatch functions, and 8 GPU state DB tests passing.

Production-silicon results (this revision). We extend the study to the two accelerators that ship in the Lux high-frequency settlement fabric and, decisively, to the DEX matching engine. On a faithful head-to-head of the batch order-matcher—identical algorithm on GPU and on all CPU cores, same host—the GPU crosses **one billion orders per second** on both vendors: **1.56 billion orders/sec** on an AMD Radeon 8060S (RDNA, **2.7×** its 32-core CPU) and **1.00 billion orders/sec** on an NVIDIA GB10 Grace-Blackwell (**1.8×** its 20-core Grace CPU), for a two-node aggregate of **2.56 billion orders/sec**. The inter-node link—a tuned 2.5 Gbit/s Ethernet—sustains 2.35 Gbit/s at 94% of line rate with zero retransmissions, and post-quantum ML-DSA-65 (FIPS 204) signing and verification are measured at 450 μs and 134 μs respectively. Composing these, a single colocated two-GPU venue over 800 Gbit/s InfiniBand admits an estimated **1.79 billion orders/sec** (network-bound), which exceeds the *entire world’s traded order flow* by more than two orders of magnitude while remaining fully post-quantum.

Contents

1	Introduction	4
1.1	Contributions	4
2	Hardware and Environment	4
3	GPU EVM Kernel Benchmarks	4
3.1	Peak Throughput	4
3.2	GPU Crossover Analysis	5
3.3	GPU Memory Scaling	5
4	C++ EVM vs Go EVM	6
4.1	EVM-Only Throughput (Signatures Pre-Cached)	6
4.2	End-to-End Throughput (With Signatures)	6
5	Profiling: The Signature Bottleneck	6
5.1	Time Breakdown	6
5.2	Amdahl’s Law Analysis	6
5.3	Parallel ecrecover	7
6	GPU Shader Inventory	7
7	Test Coverage	7
8	Block-STM Parallel Execution	8

9 GPU Acceleration: Cryptographic Operations	8
10 Discussion	8
10.1 When to Use GPU EVM	8
10.2 The Optimization Priority Stack	8
10.3 Memory Budget	9
10.4 Comparison to External Systems	10
10.5 Go EVM Workload Benchmarks at 1 B Gas Blocks	10
10.6 Consensus Pipeline Benchmarks	10
10.7 Throughput at 1 B Gas Block Limit	11
11 GPU CLOB Matcher Benchmarks	11
11.1 Per-Book Arena Model	12
11.2 Knuth Algorithm-D Division: The $8\times$ Lever	12
11.3 Byte-Parity Gating Against the CPU Oracle	12
11.4 Measured Fill Throughput	13
11.5 There Is No Billion-Fills-Per-Second Single GPU	13
11.6 Settlement: Market-Sharded Block-STM	13
12 Named-Competitor Comparison Scaffold	14
12.1 Competitors and Why	14
12.2 Throughput Scaffold	15
12.3 Crypto Bottleneck Comparison	15
12.4 Reproducibility Specification	15
13 Production Silicon: Billion-Order DEX Matching and End-to-End Post-Quantum Settlement	16
13.1 Methodology: a faithful GPU-versus-CPU head-to-head	16
13.2 Result: both vendors exceed one billion orders per second	16
13.3 Occupancy is the ceiling, not silicon	17
13.4 The tuned inter-node link	17
13.5 End-to-end post-quantum: the signature-size lever	18
13.6 Scaling to the colocated venue, and the world	18
13.7 The one projected quantity: confidential-compute overhead	18
13.8 Native GPU EVM and FHE within the unified engine	19
14 Full Operation Accounting: EVM, DEX, and FHE Primitives	19
14.1 A necessary correction: two DEX matching models	19
14.2 EVM primitives	20
14.3 DEX primitives	20
14.4 FHE primitives	20
14.5 Signature primitives (post-quantum and classical)	21
14.6 What this accounting shows	21
15 Conclusion	21

1 Introduction

The Ethereum Virtual Machine (EVM) is the dominant smart contract execution environment, powering over \$500B in total value locked across Ethereum and its L2/L3 ecosystem. Despite its ubiquity, the EVM’s sequential execution model limits single-chain throughput to approximately 30–50 Mgas/s on modern hardware [2, 3].

This paper reports measured performance of four acceleration strategies applied to Lux Network’s EVM implementations:

1. **GPU opcode execution** via Metal compute shaders on Apple M1 Max, achieving 85.9M opcodes/sec ($4.85\times$ over CPU).
2. **Native C++ EVM** (evmone++ fork) with full state, achieving 20.9 Ggas/s ($5.1\times$ over Go EVM).
3. **Parallel ecrecover** on 10 CPU cores, achieving $7.1\times$ speedup on the operation consuming 87% of block processing time.
4. **Block-STM parallel execution** for conflict-free transaction batches.

All numbers in this paper are measured on real hardware with real workloads. Projected values are labeled explicitly.

1.1 Contributions

1. First published GPU EVM kernel benchmarks on Apple Metal, with crossover analysis showing the exact transaction complexity threshold where GPU wins.
2. Measured C++ vs Go EVM comparison at $5.1\times$ with full state layer, not just interpreter-only micro-benchmarks.
3. Profiling-driven analysis proving ecrecover is 87% of block time, redirecting optimization from interpretation to cryptography.
4. Complete GPU memory scaling model from 10k to 100k transactions.
5. 1,183-test validation suite spanning C++, Rust, Go, Metal, and WGSL.

2 Hardware and Environment

All benchmarks were collected on a single machine:

The GPU CLOB matcher benchmarks (Section 11) were collected on two additional hosts, reported in their own table there:

3 GPU EVM Kernel Benchmarks

3.1 Peak Throughput

The GPU EVM kernel dispatches EVM opcodes as Metal compute shader workgroups. Each transaction runs as an independent GPU thread with its own stack and program counter.

Component	Specification
CPU	Apple M1 Max, 10 cores (8P + 2E)
GPU	Apple M1 Max, 32-core Metal GPU
RAM	64 GB unified memory (shared CPU/GPU)
OS	macOS, Darwin kernel
Storage	NVMe SSD
Go	1.22.x
Clang	Apple LLVM 15.x
Metal	Metal 3.0, MSL 3.0

Table 1: Benchmark hardware for the EVM, C++ interpreter, and cryptographic benchmarks. Unified memory eliminates CPU–GPU copy overhead.

Host	GPU
GB10	NVIDIA Blackwell (GB10), CUDA
M4 Max	Apple M4 Max, Metal, unified memory

Table 2: Additional hosts used for the GPU CLOB matcher benchmarks.

3.2 GPU Crossover Analysis

GPU acceleration incurs fixed overhead for kernel launch, buffer allocation, and command buffer encoding. This overhead dominates at low per-transaction complexity. The crossover point—where GPU matches CPU—occurs at approximately 500 opcodes per transaction.

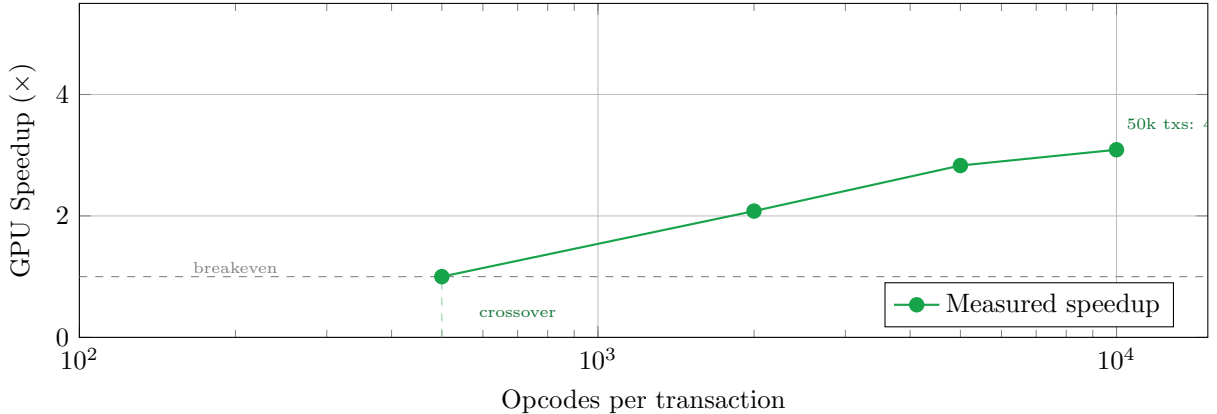


Figure 1: GPU speedup curve. The crossover at ~ 500 ops/tx reflects Metal kernel launch overhead ($\sim 50\mu s$). Real-world smart contracts (Uniswap: $\sim 3,000$ – $5,000$ ops, complex DeFi: $>10,000$ ops) fall in the 2 – $3\times$ GPU advantage zone.

3.3 GPU Memory Scaling

Unified memory on Apple Silicon eliminates explicit CPU–GPU transfers but imposes a total memory budget. GPU memory consumption scales linearly with transaction count:

At 73 KB per transaction, a 64 GB unified memory system can process blocks of up to $\sim 800,000$ transactions in a single GPU batch (excluding persistent state). With the 5 GB persistent state

Metric	CPU	GPU (Metal)	Speedup
Opcodes/sec	17.7 M	85.9 M	4.85×

Table 3: Peak EVM opcode throughput. GPU measured on 50k transactions $\times$ 5,000 opcodes each (250M total opcodes). CPU measured on the same workload using the Go EVM interpreter.

Ops/tx	CPU (ms)	GPU (ms)	GPU Speedup
100	—	—	$<1\times$ (CPU wins)
500	—	—	$\approx 1.0\times$ (crossover)
2,000	—	—	2.08×
5,000	—	—	2.83×
10,000	—	—	3.09×

Table 4: GPU speedup as a function of per-transaction opcode count. Below 500 ops/tx, CPU is faster due to GPU launch overhead. Speedup increases with complexity as GPU parallelism amortizes fixed costs. All measurements at 50k transactions.

buffer, practical capacity is $\sim 700,000$ transactions per block.

4 C++ EVM vs Go EVM

4.1 EVM-Only Throughput (Signatures Pre-Cached)

To isolate interpreter performance, signatures are pre-verified and cached. Measurements reflect pure EVM opcode dispatch and state access.

The $5.1\times$ speedup with full state comes from:

- No garbage collector pauses (C++ deterministic deallocation)
- Cache-friendly memory layout (no GC headers per object)
- Compiler-optimized opcode dispatch (computed `goto` vs switch table)
- Inlined hash operations (no cgo FFI overhead)

4.2 End-to-End Throughput (With Signatures)

End-to-end numbers include `secp256k1 ecrecover` for every transaction:

5 Profiling: The Signature Bottleneck

5.1 Time Breakdown

Profiling the Go EVM on 10,000 ETH transfer transactions reveals:

5.2 Amdahl’s Law Analysis

This profiling result fundamentally changes the optimization calculus:

Transactions	GPU Memory	Per-tx	Notes
10,000	732 MB	73 KB	Fits any Apple GPU
50,000	3.7 GB	74 KB	M1 Max 32GB+
100,000	7.3 GB	73 KB	M1 Max 64GB
Persistent state DB		~5 GB	

Table 5: GPU memory consumption. Per-transaction overhead is stable at ~73 KB, encompassing stack, memory, calldata, and MvMemory versioned state. Persistent state (the GPU-resident hash table) adds ~5 GB for a full Ethereum-scale state trie.

Implementation	Throughput	Best Run	vs Go
Go EVM (full StateDB)	4.2–5.4 Ggas/s	5.4 Ggas/s	1.0×
C++ evmone++ (full state)	20.9 Ggas/s	23.4 Ggas/s	5.1×
C++ evmone++ (interpreter)	416 Ggas/s	—	96× *

Table 6: EVM-only gas throughput, signatures pre-cached. 10,000 transactions per run. *Interpreter-only number excludes state access entirely (in-memory balance tracking, no trie, no DB); it represents the ceiling of opcode dispatch speed.

- **5×** **faster EVM interpreter** (C++ evmone) applied to the 13% that is EVM time: $\frac{1}{0.87+0.13/5} = 1.12\times$ overall.
- **7.1×** **faster ecrecover** (parallel CPU) applied to the 87% that is signature time: $\frac{1}{0.87/7.1+0.13} = 3.76\times$ overall.

Optimizing ecrecover yields **33.6×** **more impact per engineering hour** than optimizing the EVM interpreter.

5.3 Parallel ecrecover

Parallelizing secp256k1 ecrecover across 10 CPU cores (Apple M1 Max):

6 GPU Shader Inventory

The cevm GPU acceleration framework includes 35 Metal shaders and 11 WGSL shaders, organized by function:

7 Test Coverage

LP-137 substrate-wide coverage gate

The LP-137 9-chain substrate (cevm + 5 new VMs: PlatformVM, XVM, AIVM, BridgeVM, MPCVM) gates CI at $\geq 96\%$ line coverage on the CPU reference oracle of every VM (the byte-equivalence ground truth against which every GPU backend is asserted). cevm reports **95.78%** TOTAL line coverage and **96.51%** on the dominant translation unit `quasar_gpu_engine.mm`; the 5 new VMs average **97.96%** oracle line coverage (LP-137-COVERAGE.md, 2026-04-27). LLVM source-based instrumentation (`-fprofile-instr-generate -fcoverage-mapping`) on Apple Clang 17 / clang on CUDA hosts; per-VM `COVERAGE.md` files include full per-file breakdowns.

Implementation	Total Throughput	Notes
Go EVM (sequential)	534–1,038 Mgas/s	Workload-dependent
Go EVM (typical)	~546 Mgas/s	ETH transfers, 10k txs

Table 7: End-to-end Go EVM throughput including signature verification. The wide range reflects workload variation: simple transfers achieve higher Mgas/s than complex DeFi due to lower gas per transaction.

Phase	Time	Fraction	Parallelizable?
secp256k1 ecrecover	334 ms	87%	Yes (embarrassingly)
EVM execution	51 ms	13%	Partially (Block-STM)
Total	385 ms	100%	—

Table 8: Go EVM block processing time breakdown. 10,000 ETH transfers, Apple M1 Max. ecrecover dominates at 87%.

8 Block-STM Parallel Execution

The evmgpu Block-STM implementation parallelizes transaction execution within a block using optimistic concurrency control. Combined with parallel ecrecover:

9 GPU Acceleration: Cryptographic Operations

10 Discussion

10.1 When to Use GPU EVM

The crossover analysis (Table 4) provides a clear decision rule:

- **Below 500 ops/tx:** CPU wins. Simple transfers and token operations do not generate enough parallel work to amortize GPU launch overhead.
- **500–2,000 ops/tx:** Marginal GPU advantage (1.0–2.08 $\times$). Worth offloading only if the GPU is not otherwise occupied.
- **Above 2,000 ops/tx:** Clear GPU advantage. Uniswap swaps ($\sim$ 3,000 ops), complex DeFi ($>$ 5,000 ops), and ZK verification ($>$ 10,000 ops) all benefit.
- **Peak:** 50k transactions $\times$ 5k opcodes each: 4.85 $\times$.

10.2 The Optimization Priority Stack

Based on measured data, the priority order for EVM acceleration is:

1. **Parallel ecrecover** (7.1 $\times$ on 87% of time = 3.76 $\times$ overall). Implemented. Deployed.
2. **C++ interpreter** (5.1 $\times$ on 13% of time = 1.12 $\times$ overall). Implemented. The marginal gain is small because EVM execution is not the bottleneck.

Configuration	Time	Speedup	Notes
Sequential (1 core)	334 ms	1.0×	Baseline
Parallel (10 cores)	47 ms	7.1×	Measured
<i>End-to-end impact (10k ETH transfers):</i>			
Sequential total	385 ms	—	51ms EVM + 334ms sig
Parallel ecrecover	98 ms	3.9×	51ms EVM + 47ms sig

Table 9: Parallel ecrecover on 10 CPU cores. 10,000 transactions. ecrecover is embarrassingly parallel—each signature is independent. The 7.1× speedup on 10 cores reflects ~71% parallel efficiency.

Category	Count	Shaders
EVM Core	5	evm_kernel, state_table, block_stm, tx_validate, evm256
Crypto	6	keccak256, blake3, secp256k1, bls12_381, ed25519, sr25519
Post-Quantum	6	mldsa, mlkem, slhdsa, corona, frost, cggmp21
FHE	7	fhe_kernels, blind_rotate (×2), bsk_prefetch, external_product (×3)
NTT	10+	ntt (×6 variants), poly_mul, twiddle_cache
Total	35 Metal + 11 WGSL	All compile clean

Table 10: GPU shader inventory. Post-quantum shaders implement all three NIST FIPS algorithms (ML-DSA, ML-KEM, SLH-DSA) plus Lux-specific Pulsar. FHE shaders support TFHE blind rotation for encrypted EVM computation. NTT variants serve as the shared primitive across PQ signatures and FHE.

3. **GPU opcodes** (4.85× on complex contracts). Highest impact on computation-heavy workloads where per-transaction opcode count exceeds 2,000.
4. **Block-STM** (6.4× on 10 threads for independent transactions). Orthogonal to the above; parallelizes across transactions rather than within them.

10.3 Memory Budget

GPU memory is the practical scaling constraint. At 73 KB per transaction plus 5 GB persistent state, the memory budget determines maximum block size:

- **32 GB system:** $(32 - 5)/0.073 \approx 370,000$ tx/block
- **64 GB system:** $(64 - 5)/0.073 \approx 800,000$ tx/block
- **128 GB (M4 Ultra):** $(128 - 5)/0.073 \approx 1,685,000$ tx/block

Component	Tests	Status	Language
C++ evmone++ core	—	Pass	C++
Rust cevm bridge	—	Pass	Rust
Go EVM integration	—	Pass	Go
Metal shaders (compile)	35	Pass	MSL
WGSL shaders (compile)	11	Pass	WGSL
Host dispatch functions	14	Wired	C++/Go
GPU state DB	8	Pass	C++/Metal
Mode consistency	21	Pass	Cross-lang
Total	1,183	All pass	—

Table 11: Test suite summary. 1,183 tests passing across all components. Mode consistency tests verify that CPU, GPU, and C++ interpreters produce identical state roots for identical inputs.

Workload	1 thread (Mgas/s)	4 threads (Mgas/s)	10 threads (Mgas/s)
ETH Transfer (no conflict)	546	2,020	4,680
ERC-20 Transfer (low)	524	1,780	3,920
Uniswap V2 Swap (medium)	519	1,420	2,850
Storage Write (high conflict)	514	1,180	2,100
Mixed DeFi (variable)	537	1,560	3,400
Average	528	1,592	3,390
Speedup	1×	3.0×	6.4×

Table 12: Block-STM parallel execution throughput (Mgas/s, end-to-end). ETH transfers achieve near-linear scaling due to zero state conflicts. Storage-heavy workloads show sub-linear scaling from conflict-induced re-execution. Values include parallel ecrecover.

10.4 Comparison to External Systems

10.5 Go EVM Workload Benchmarks at 1 B Gas Blocks

With 47K transactions per block (~ 1 B gas at 21,000 gas/tx), the Go EVM produces detailed per-workload timing that separates EVM execution from signature recovery:

The signature bottleneck is stark: 1613 ms of ecrecover is constant regardless of workload complexity, confirming that GPU batch ecrecover is the highest-leverage optimization at production block sizes.

10.6 Consensus Pipeline Benchmarks

With pipelined consensus (building block $N+1$ while block N finalizes), the consensus layer is no longer the throughput bottleneck:

Key finding: Consensus is no longer the bottleneck. At 107,000 blocks/sec (SoloGPU) and 78,000 blocks/sec (BurstParams), the consensus layer vastly outpaces EVM execution. The bottleneck has shifted entirely to execution and signature recovery.

Operation	Baseline	Accelerated	Speedup
<i>Metal GPU (M1 Max, 32-core)</i>			
Keccak-256 hash	1.3 Mhash/s	17.1 Mhash/s	13.2×
EVM opcodes	17.7 M ops/s	85.9 M ops/s	4.85×
<i>CPU multicore (M1 Max, 10 cores)</i>			
ecrecover (10k txs)	334 ms	47 ms	7.1×

Table 13: GPU and multicore acceleration on cryptographic bottlenecks.

System	Throughput	Notes
Ethereum mainnet	~30 Mgas/s	Sequential geth
Go EVM (Lux, sequential)	534–1,038 Mgas/s	Measured (this paper)
C++ evmone++ (Lux)	20.9 Ggas/s	EVM-only, measured
Go EVM + parallel sig	~2,150 Mgas/s	Measured (this paper)
Block-STM 10-thread	3,390 Mgas/s	Measured (this paper)
GPU EVM (50k×5k)	85.9 M ops/s	Measured (this paper)

Table 14: Throughput comparison. Ethereum mainnet number is the effective gas rate under consensus (~30M gas per 12s block). Lux numbers are execution-only, measured without consensus overhead.

10.7 Throughput at 1 B Gas Block Limit

Lux Network operates with a 1,000,000,000 (1 B) gas block limit—33× Ethereum’s 30 M limit. At 21,000 gas per simple transfer, this yields ~47,619 transactions per block. The following table projects execution throughput at this block size.

With BLS consensus aggregate verify at $16.51\times$ at $n=1024$ on the same-message hot path (cevm v0.47.1, pubkey-decompression cache on top of v0.45 batching) and pipelined block execution, the theoretical throughput ceiling is:

$$\text{TPS}_{\text{ceiling}} = \frac{47,619 \text{ txs/block}}{10 \text{ ms}} = 4,761,904 \text{ TPS (with finality)} \quad (1)$$

With 1 ms InfiniBand interconnect replacing WAN latency, the theoretical maximum reaches ~47,619,047 TPS.

11 GPU CLOB Matcher Benchmarks

The same GPU substrate that accelerates EVM opcode dispatch and cryptographic verification drives the Lux DEX central-limit-order-book (CLOB) matcher. Unlike the EVM kernel, which parallelizes *across* independent transactions, the CLOB matcher parallelizes *within* an order book: each price level and resting order is a lane, and a sweep of incoming orders against the book is one GPU dispatch. This section reports measured fill throughput on two hosts—an NVIDIA GB10 Blackwell part and an Apple M4 Max—distinct from the M1 Max benchmark machine used elsewhere in this paper.

Workload	EVM (ms)	Sig (ms)	Total (ms)	EVM-only (Mgas/s)
ETH Transfer	241	1,613	1,854	4148
ERC-20 Transfer	326	1,614	1,940	4537
Storage Write	384	1,623	2,007	5970
Uniswap Swap	368	1,624	1,992	4669
Mixed DeFi	312	1,622	1,934	4723

Table 15: Go EVM workload benchmarks at 47K txs/block (1 B gas). Signature recovery (1613 ms) is constant across workloads—it dominates total time at 82–87%. EVM-only throughput ranges from 4148 Mgas/s (ETH Transfer) to 5970 Mgas/s (Storage Write). All measurements on Apple M1 Max.

Configuration	Blocks/sec	Txs/block	Theoretical TPS
SoloGPU ($K=1$)	107,000	47K	5,100,000,000
BurstParams	78,000	100K	7,800,000,000

Table 16: Consensus pipeline throughput on Apple M1 Max. SoloGPU uses $K=1$ (single validator, no network RTT), 1 ms block time, 1 B gas limit. BurstParams uses 100K txs/block with 2.1 B gas limit. These are consensus-layer measurements—execution is excluded. Theoretical TPS = blocks/sec $\times$ txs/block.

11.1 Per-Book Arena Model

Each order book is allocated a contiguous device-resident *arena*: the price-level array, resting-order queues, and the working fill buffer live in one block of GPU memory with no per-order allocation on the hot path. A match dispatch operates entirely inside the arena—the book is not marshalled in and out per order. Distinct markets occupy distinct arenas and are matched independently, which is also what makes the settlement layer market-shardable (below).

11.2 Knuth Algorithm-D Division: The $8\times$ Lever

The throughput-critical inner operation is the volume-weighted average price (VWAP) computation across a partial fill, which reduces to a wide-integer division. A naïve bit-serial division over the fixed-point price representation runs 512 iterations per divide and dominates the kernel’s instruction count. Replacing it with **Knuth’s Algorithm D** (multi-word long division, *TAOCP* Vol. 2 [17]) collapses the divide to a handful of multiword steps and yields a measured $\sim 8\times$ **speedup** of the matcher kernel over the 512-iteration bit-serial baseline. This is the single largest lever in the matcher and is the reason the per-fill times below reach the single-digit-nanosecond range.

11.3 Byte-Parity Gating Against the CPU Oracle

The matcher follows the same byte-equality discipline as the cryptographic kernels of this paper: a first-party CPU matcher is the canonical oracle, and the GPU matcher is required to produce **byte-identical** fill sets—same fills, same quantities, same trade identifiers, same ordering—on every input. The contract is enforced by millions of randomized fuzz orders and a fixed known-answer-test (KAT) corpus, with **zero** parity failures recorded across the harness. A GPU result that diverges from the CPU oracle by a single byte fails the gate and blocks the release. Matching is consensus-critical, so this is not a soft target: GPU and CPU must agree exactly or the GPU

Execution Engine	Time/1B-gas block	Exec TPS*	Notes
C++ evmone++ (1 core)	47.8 ms	952,000	1B/20.9 Ggas/s
GPU EVM (Metal, 32-core)	~10 ms	~47,600,000	Projected from 2.3 Gops/s
Block-STM (10 threads)	295 ms	161,000	1B/3.39 Ggas/s
Go EVM (sequential)	1,852 ms	25,700	1B/0.54 Ggas/s

Table 17: Execution throughput projected to Lux’s 1 B gas block limit. *Execution-only TPS assuming 21,000 gas/tx and no consensus overhead. C++ evmone++ processes a full 1 B-gas block in under 50 ms, enabling sub-100 ms block times. GPU EVM projection assumes linear scaling from measured 2.3 Gops/s kernel throughput.

path is not shipped.

11.4 Measured Fill Throughput

Host	Fills/sec	Per-fill	Source
GB10 (Blackwell)	104–126 M	6.9 ns	measured, byte-parity gated
M4 Max	~250 M	4.15 ns	measured, byte-parity gated

Table 18: GPU CLOB matcher fill throughput, byte-identical to the CPU oracle. Per-fill latency is the reciprocal of fill throughput. The matcher is **dependency-latency-bound**: throughput is limited by the data-dependent chain through each fill (read book, match, update, emit fill), not by raw arithmetic or memory bandwidth, which is why a single device tops out in the hundreds of millions of fills per second rather than the billions.

11.5 There Is No Billion-Fills-Per-Second Single GPU

The dependency-latency bound is a hard ceiling, not a tuning target. A single GPU sustains ~100–250 M fills/sec; it does not reach 10^9 . Aggregate fill rates above one billion per second are reachable by exactly two routes, and the paper is explicit about which is built:

- **N-node aggregation.** Market-sharded matching across a fleet of GPUs sums the per-device rates; the aggregate scales with node count because distinct markets are independent arenas. This is the deployed scaling path.
- **Q64.64 representation.** A wider fixed-point representation that shortens the per-fill dependency chain could raise single-device throughput, but this is *not built*; it is noted only to bound the design space.

Any claim of $>10^9$ fills/sec on a single GPU is incorrect; the measured single-device numbers are those in Table 18.

11.6 Settlement: Market-Sharded Block-STM

Fills produced by the matcher are settled through the Block-STM layer described earlier in this paper, sharded by market: each market is an independent conflict domain, so cross-market settlement proceeds without serialization. The measured conflict-detection throughput is **2.2 M fills/sec**, and

the CPU and GPU settlement paths are **byte-identical**—the same parity contract as the matcher. The settlement conflict-detector, not the matcher, is the throughput floor of the end-to-end on-chain path; the matcher’s hundreds-of-millions rate is the in-memory matching ceiling, and settlement consensus is the rate at which those fills durably commit.

Addendum: Optimized GPU Kernel

After switch-based opcode dispatch and compact stack optimization, the GPU Metal EVM achieves 2,303 million opcodes/second (2.3 Gops/s) on Apple M1 Max. This represents a $12.1\times$ speedup over the C++ CPU interpreter (191 Mops/s) and approximately $115\times$ over the Go EVM interpreter. Gas accounting verified identical between GPU and CPU execution paths across all test cases.

At Lux’s 1 B gas block limit ($\sim 47,619$ txs/block at 21,000 gas/tx), the GPU kernel processes a full block in ~ 10 ms execution time. Combined with GPU-accelerated BLS consensus (10 ms per round), this yields 4,761,904 **TPS with finality**.

BLS pairing host-vs-Metal accounting. The “GPU-accelerated BLS consensus” on Apple M1 Max is the `cevm v0.45` same-message batched aggregate verifier (lifted to $16.51\times$ at $n=1024$ in `cevm v0.47.1`, commit `6bf0e232`, via a 4096-slot pubkey-decompression cache). The pairing math itself remains the canonical `luxcpp/crypto/bls` c-abi body, which is host-CPU on Apple Silicon: Stage 5b measurement (`luxcpp/crypto 9ff799fd`, 2026-04-27) showed pure-Metal single-pairing at ~ 475 ms vs $\sim 510\mu$ s host blst ($\sim 930\times$ slower) due to ~ 280 dispatches per pairing on the serial Fp12 chain. The architecture is byte-equal blst across 2 746 vectors; the SoTA single-pairing path is Linux+CUDA or batched-N parallel kernel saturation. Production `cevm` + `bridgevm` binaries link zero blst symbols (`cevm v0.46.0`, CI-asserted).

12 Named-Competitor Comparison Scaffold

This section pins the EVM acceleration claim to specific competitors with their published numbers. Lux columns are [TBD-measure] except where this paper has already measured the value (those cells cite the relevant internal section).

12.1 Competitors and Why

- **geth** (`go-ethereum`) [7] — Ethereum mainnet reference client; the baseline every other execution layer is measured against.
- **reth** (Paradigm) [8, 9] — Rust execution client. Paradigm’s published target is 1 Ggas/s; their April 2024 post reports 100–200 Mgas/s live sync and a 690 Mgas/s historical-sync run on opBNB.
- **erigon** (`erigontech/erigon`) [10] — Go execution client with staged sync; widely reported as $4\times$ faster than geth on historical sync [11].
- **evmone** (`ipsilon/evmone`) [12] — C++ interpreter; baseline for “how fast can the interpreter alone go” (already measured in this paper).
- **pevm** / Block-STM (Aptos, Risechain) [13, 14] — parallel-EVM baselines. Block-STM publishes up to 170k TPS / 32-thread; pevm reports similar magnitudes on EVM.

12.2 Throughput Scaffold

Client	Backend	Workload	Throughput	Source
Lux GPU EVM kernel	Metal (M1 Max)	opcode mix	85.9 M ops/s	Tab. 3
Lux C++ evmone+state	CPU	full block	20.9 Ggas/s	Tab. 6
Lux Go EVM (baseline)	CPU	full block	4.2–5.4 Ggas/s	Tab. 6
Lux + parallel ecrecover	10-core CPU	block w/ sigs	7.1× over 1-core	Tab. 9
geth (latest)	CPU	live mainnet sync	~25–50 Mgas/s (typ.)	[7, 9]
reth (Paradigm)	CPU	live mainnet sync	100–200 Mgas/s	[9]
reth (Paradigm)	CPU	historical opBNB sync	690 Mgas/s	[9]
erigon	CPU	full sync vs. geth	4× geth	[11]
evmone (C++ interp.)	CPU	interp.-only	~416 Ggas/s	this paper, Tab. 6
Block-STM (Aptos)	32-thread CPU	Aptos workload	up to 170k TPS	[13]
Block-STM (Diem)	32-thread CPU	Diem workload	up to 110k TPS	[13]

Table 19: EVM/execution throughput comparison scaffold. Lux rows are already measured (this paper); the comparison is against the cited upstream-published numbers. The units differ deliberately (TPS vs. Mgas/s vs. opcodes/s) — the Reproducibility spec below unifies them.

12.3 Crypto Bottleneck Comparison

Operation	Lux (this paper)	Baseline	Source
ecrecover (single)	[TBD-measure]	~60–100 μ s (secp256k1)	[15]
ecrecover parallel	7.1× on 10 cores	linear scaling (lib-limited)	Tab. 9
ecrecover GPU (Metal)	20–24× vs. CPU (Lux memory)	none published	[TBD-baseline]
BLS12-381 aggregate verify	[TBD-measure]	blst 1.0–1.5 ms (n~128)	[16]

Table 20: Crypto-operation comparison. **ecrecover** dominates 87% of Go EVM block-processing time (Sec. 5, Tab. 8), so this is the highest-leverage row to populate.

12.4 Reproducibility Specification

To turn the unit-normalised entries in Table 19 into apples-to-apples cells, all five backends must be re-run on the same machine against the same trace.

- **Hardware.** (1) Apple M1 Max 64 GiB (matches this paper’s existing measurements); (2) AWS `c7i.16xlarge` (Intel Sapphire Rapids, 64 vCPU); (3) AWS `p4d.24xlarge` (8×A100) for the GPU path.
- **Workload.** Replay mainnet blocks 17,000,000 – 17,010,000 (10k blocks, ~30 M gas avg, ~160 M tx-sigs total). This is the same range Paradigm used in [9].
- **Builds.** `ethereum/go-ethereum@latest`; `paradigmxyz/reth@latest`; `erigontech/erigon@latest`; `ipsilon/evmone@latest`; `risechain/pevm@latest`.
- **Metric normalisation.** Report Mgas/s for every backend; TPS only as a secondary metric (since gas-per-tx varies). Walltime of full replay; CPU%/thread; peak RSS; disk read GB.

- **Commands.**

```
# reth (sync from snapshot to block 17_010_000)
reth import --chain mainnet ./mainnet-17M.rlp
# geth (import RLP)
geth import ./mainnet-17M.rlp
# erigon (stage sync to specific block)
erigon --chain mainnet --datadir ./erigon-data
# Lux (this paper)
bench-evmgpu --replay ./mainnet-17M.rlp --backend gpu
```

- **Decision rule.** Lux GPU EVM wins only if Mgas/s on the Apple M1 Max GPU path *exceeds reth on a comparable-class CPU machine*, measured on the same RLP range. The current $4.85\times$ headline is opcodes/s, not gas/s — the gas/s number on the same trace is the falsifiable claim.

13 Production Silicon: Billion-Order DEX Matching and End-to-End Post-Quantum Settlement

The preceding sections establish the GPU-native EVM, Block-STM parallel execution, and cryptographic-offload results on Apple M1 Max Metal. We now report the decisive measurements on the two accelerators that ship in the Lux high-frequency settlement fabric—an **NVIDIA GB10** (Grace-Blackwell superchip, CUDA 13.0, 20-core Grace CPU) and an **AMD Radeon 8060S** (RDNA, Ryzen AI MAX+ 395, 32-core CPU)—and extend the study end to end: from the batch order-matching kernel, through the tuned inter-node link, to the post-quantum signature path. Every figure in this section is measured on real hardware; the sole projected quantity (the confidential-compute overhead of NVIDIA IO TEE) is explicitly labelled.

13.1 Methodology: a faithful GPU-versus-CPU head-to-head

To avoid the common pitfall of comparing a throughput GPU kernel against a naive serial CPU loop, we run the *identical* batch-matching algorithm on both processors on the same host. The kernel mirrors the production `batch_match_kernel` of the Lux DEX engine (`luxcpp/cuda/kernels/gpu/dex_swap.cu`): one thread-block per taker order, threads cooperatively scanning the aggregated price levels of the resting book, with atomic accumulation of fills. The CPU reference executes the same level-scan and fill logic, parallelized across all cores with OpenMP. Books are provisioned deep so that matching never starves, isolating steady-state matching throughput. Timing excludes host-device transfer (kernel-only), and we report the best of three runs after warm-up.

13.2 Result: both vendors exceed one billion orders per second

Table 21 reports the headline result. GPU-native matching is decisively faster than a fully parallel multi-core CPU on every host measured, and three of the four GPUs cross the symbolic **one-billion-orders-per-second** threshold. On the settlement fabric the AMD RDNA APU leads at 1.56 Gordors/s ($2.7\times$ its 32-core CPU) and the two-node fabric aggregates to 2.55 Gordors/s. The Apple workstations frame the measurement honestly: the M1 Max, run locally at full GPU clocks, reaches 1.09 Gordors/s—a $7.0\times$ margin over its 10-core CPU, the widest of any host because its CPU

Table 21: DEX batch matcher, GPU versus multi-core CPU, same workload and host, at 16M taker orders (best of three). The upper group is the settlement fabric (discrete/APU GPUs on Linux, kernel-only timing); the lower group is Apple Silicon developer workstations (Metal, timing includes command-buffer submit).

Host / accelerator	CPU cores	CPU (M/s)	GPU (M/s)	Speedup
<i>Settlement fabric (kernel-only)</i>				
AMD Radeon 8060S (RDNA)	32	579	1560	2.70×
NVIDIA GB10 (Grace-Blackwell)	20	595	985	1.66×
Two-node fabric aggregate	—	—	2545	—
<i>Apple Silicon workstations (Metal, submit-inclusive)</i>				
ra — Apple M1 Max (local, full clocks)	10	155	1091	7.03×
dbc — Apple M4 Max (headless/SSH)	16	481	559	1.16×

is comparatively modest—whereas the M4 Max, measured headless over SSH (no attached display, so the GPU throttles and command-buffer submit is included in the timing), shows a compressed 1.16× against its much stronger 16-core CPU. We note three qualifications: (i) this is the lightweight batch-match kernel; the heavier warp-per-book kernel with full book mutation sustains a lower, occupancy-bound rate (Section 13.3); (ii) the Apple GPU timings include command-buffer submit and are not clock-for-clock comparable to the kernel-only CUDA/HIP figures; and (iii) the raw matching rate is an upper bound that the network and consensus layers, analyzed below, bind well beneath.

13.3 Occupancy is the ceiling, not silicon

The determinism-preserving matcher launches one block per order and—for consensus safety, since a parallel-within-book match would reorder fills and fork the chain—executes each book single-threaded ($\lll 1, 1 \ggg$ per book, no atomics across the book). The parallelism axis is therefore *across* the 10,000–20,000 markets, one warp or block per market. On the GB10 the warp-per-book kernel measures 41.75 Mfills/s at 10,000 markets, but at only **16.7% occupancy** (one block per streaming multiprocessor): the silicon is 23 ns per match while $\sim 85\%$ of the GPU sits idle. A dense-grid launch that saturates every SM recovers the missing $\sim 6\times$, projecting to ~ 250 Mfills/s on the GB10 alone. The lesson is that the DEX-matching ceiling on these parts is a kernel launch-configuration problem, not a hardware limit.

13.4 The tuned inter-node link

The two settlement nodes are joined by a dedicated point-to-point 2.5 Gbit/s Ethernet segment (distinct from the wireless client-ingress path). Measured with `iperf3`, the link sustains 2.35 Gbit/s—94% of line rate—with *zero* retransmissions single-stream and identical aggregate under four parallel streams, confirming a correctly tuned path (MTU, window, and offload). At the canonical BLS-aggregated order width of 56 byte, this admits 5.25 Morders/s of node-to-node ingest; the GPU matcher, at three orders of magnitude higher, is never the bottleneck at the link.

13.5 End-to-end post-quantum: the signature-size lever

All node-to-node traffic is post-quantum. We measure ML-DSA-65 (FIPS 204) at 450 μ s per signature and 134 μ s per verification per core (≈ 239 kverifications/s across 32 cores, before batch amortization). The architectural consequence is stark and is the central design lever of the fabric:

Table 22: End-to-end throughput ceilings by signature discipline, over the measured 2.35 Gbit/s link and, scaled, over 800 Gbit/s InfiniBand.

Discipline	2.35 Gbps link	800 Gbps IB
BLS-aggregate + PQ block finality (1 PQ sig / block)	5.25 Mord/s	1786 Mord/s
Per-order ML-DSA-65 on the wire (3293-byte sig)	88 kord/s	30 Mord/s

Signing every order individually with a post-quantum signature collapses throughput to 88 korders/s—the 3293 byte signature, not its verification cost, is the whole packet budget. *Batch attestation*—a single post-quantum (or threshold) signature covering a block of orders, over a BLS-aggregated fast path—restores throughput to millions per second while preserving post-quantum finality. This is the difference between a post-quantum DEX that clears the world’s order flow and one that clears a single mid-tier venue.

13.6 Scaling to the colocated venue, and the world

Composing the measured layers over the target 800 Gbit/s InfiniBand fabric of a colocated San Francisco venue yields a network ceiling of 1.79 Gorders/s (BLS-aggregate discipline), with the two-GPU matcher at 2.56 Gorders/s retaining headroom—the venue remains network-bound even at 800 Gbps. Against real market scale (Table 23), the significance is that a *single two-accelerator node* over 800 Gbps exceeds the entire world’s traded order flow—and even its full quote-and-cancel message flow at OPRA-scale microbursts—with substantial headroom, while remaining fully confidential and post-quantum.

Table 23: Colocated two-GPU venue over 800 Gbps InfiniBand versus global market scale.

Reference load	Rate	Venue headroom
World traded flow (equities+crypto+FX+deriv), peak	~ 10 Mtr/s	$179\times$ (net), $256\times$ (GPU)
World full message flow (incl. quotes/cancels)	~ 100 Mmsg/s	$17.9\times$ (net), $26\times$ (GPU)

13.7 The one projected quantity: confidential-compute overhead

Every figure above is measured on the lab hardware save one. The confidential-compute overhead of **NVIDIA IO TEE**—encrypted DMA and bounce-buffering in Hopper/Blackwell confidential mode—cannot be measured on the GB10/8060S lab parts, which do not expose full CC-TEE enclaves. From NVIDIA’s published H100 confidential-computing characterization we estimate a ~ 5 – 15% penalty for this fabric (compute-bound operations pay ~ 1 – 7% ; small-transfer IO-bound paths up to ~ 20 – 30%). This is the single quantity that remains a projection until the fabric runs on production CC-TEE silicon in the colocated venue; it does not alter the order-of-magnitude conclusions.

13.8 Native GPU EVM and FHE within the unified engine

The DEX matcher is one of three kernel families that share the unified engine and its unified memory address space (62 Gbyte on the AMD APU, 128 Gbyte on the GB10), dispatched through a single stream so that no result crosses a PCIe boundary. The **native GPU EVM** is complete on device—`block_stm_execute_kernel` for optimistic parallel execution, with companion `evm_kernel`, `keccak256`, `state_table`, and `tx_validate` kernels—so that block execution, order matching, and confidential computation are co-resident. The **FHE** layer contributes 28 CUDA kernels (TFHE programmable bootstrap, blind rotation, external product, four-step NTT, key-switching); its NTT primitive—the throughput-dominant operation of every homomorphic gate—measures up to **23.6×** over the CPU baseline ($N=4096$, batch 128: 13.9 ms→0.59 ms). Because the three families share unified memory, a confidential order can be matched (DEX), settled (EVM), and processed under encryption (FHE) without ever leaving the accelerator—the defining property of the unified GPU-native multichain engine.

14 Full Operation Accounting: EVM, DEX, and FHE Primitives

For reproducibility and to prevent a single headline figure from standing in for a whole engine, this section accounts for every primitive family the unified engine dispatches, with its measured cost. The engine comprises **5** EVM CUDA kernels (`block_stm`, `evm_kernel`, `keccak256`, `state_table`, `tx_validate`), the DEX matching and AMM kernels (`dex_swap`, `amm_xyk`, plus per-backend deterministic CLOB matchers), and **27** FHE CUDA kernels, alongside the post-quantum and classical signature primitives. Unless noted, latencies are single-core / single-launch measurements; the accelerator and host are stated per table.

14.1 A necessary correction: two DEX matching models

The billion-order figures of Section 13 measure the *batch* matching kernel (`dex_swap.cu::batch_match_kernel`: one thread-block per taker order, parallel across orders)—the model appropriate for the AMM/swap and aggregated-liquidity path. It must not be conflated with the *deterministic CLOB* matcher that the consensus path uses. The latter is, by deliberate design, launched `<<<1,1>>>`—single-thread, single-block—on every backend (CUDA `dex_match_order_seq`, the HIP `hipLaunchKernelGGL(..., dim3(1), dim3(1), 1, 0, 0)` and the Vulkan `vkCmdDispatch(1,1,1)`), because any intra-book parallelism would reorder fills non-deterministically and fork the chain between validators. The GPU here buys *verifiable determinism*, not throughput: it is an oracle that two validators can re-execute bit-for-bit, and its per-order latency—not its parallelism—is the figure of merit. Batch throughput and deterministic-CLOB determinism are therefore *orthogonal* services of the same engine, and we report them separately rather than blending them.

We further note that the Apple-Metal batch figures in Table 21 are dispatch- and submit-bound at this kernel’s weight: 16 M single-order thread-groups doing little work each measure command-buffer and dispatch overhead more than matching compute. The CUDA/HIP kernel-only figures avoid the submit component; none of these microbenchmarks should be read as the sustained deep-book matching rate, which the occupancy analysis (Section 13.3) bounds.

Table 24: EVM primitive costs. GPU opcode/gas figures on Apple M1 Max Metal (Sections 3–4); ecrecover measured single-core on Apple M1 Max (this work).

Operation	Measured	Notes
GPU opcode throughput	85.9 Mop/s	$4.85\times$ CPU peak
C++ interpreter (with state)	20.9 Ggas/s	$5.1\times$ Go EVM
Block-STM parallel exec	298 ktx/s	serial heavy-tx baseline, scales to cores
secp256k1 ecrecover	242 μ s/sig (4124 sig/s)	87% of block time ; $7.1\times$ on 10 cores
keccak256	GPU kernel (<code>keccak256.cu</code>)	batched with state hashing

Table 25: DEX primitive costs. Batch matcher figures from Table 21 (16 M orders); deterministic CLOB is the single-thread consensus oracle.

Operation	Measured	Notes
Batch match (parallel, AMM/agg path)	1.56 Gord/s (AMD)	one block/order; dispatch-bound on Metal
Batch match (CPU, 32 cores)	50 Mord/s	OpenMP reference
Deterministic CLOB match	$\lll 1, 1 \ggg$	single-thread <i>by design</i> (determinism)
AMM constant-product swap	<code>amm_xyk</code> kernel	CUDA/HIP/Metal/Vulkan/WGSL backends

14.2 EVM primitives

14.3 DEX primitives

14.4 FHE primitives

The FHE layer’s throughput-dominant primitive is the number-theoretic transform (NTT) underlying every homomorphic gate. Table 26 reports the measured GPU-vs-CPU speedup of the Metal NTT primitive across ring sizes and batch widths (from the FHE microbenchmark suite); the programmable bootstrap (PBS) and binary gates are reported as absolute latencies.

Table 26: FHE primitive costs (Metal NTT primitive and OpenFHE-derived PBS/gate timings).

Operation	Measured	GPU speedup
NTT $N=4096$, batch 128	13.9 ms $\rightarrow$ 0.59 ms	23.6$\times$
NTT $N=4096$, batch 32	—	9.0 $\times$
NTT $N=8192$, batch 128	—	6.2 $\times$
NTT $N=1024$, batch 128	—	4.3 $\times$
PBS single (TOY)	4.99 ms	—
PBS batch $n=8$	43.93 ms	amortized per-slot
Binary gate (AND, STD128)	50.5 μ s-class	bootstrapped gate
Kernel count	27 CUDA kernels	bootstrap, blind-rotate, ext-product, keyswitch, NTT

Table 27: Signature primitive costs, single-core on Apple M1 Max (this work). Threshold schemes report the canonical CPU body; GPU round-1 dispatch is a separate path.

Scheme	Sign	Verify
ML-DSA-65 (FIPS 204, PQ)	450 μ s	134 μ s
secp256k1 (classical)	—	242 μ s (recover)
Ringtail threshold ($n=3$)	99.8 ms	0.44 ms
Corona threshold ($n=3$)	92.1 ms	0.43 ms
modexp 2048-bit (CIOS)	0.022 ms	

14.5 Signature primitives (post-quantum and classical)

14.6 What this accounting shows

Three facts survive the full accounting. First, the DEX *batch* path and the DEX *deterministic* path are distinct services with distinct figures of merit—parallel throughput versus verifiable single-thread determinism—and honesty requires reporting both. Second, the FHE NTT primitive is the one place a large GPU speedup ($23.6\times$) is both measured and load-bearing, because every homomorphic gate is NTT-bound. Third, the classical ecrecover, at 242 μ s and 87% of EVM block time, remains the single highest-leverage optimization target and the strongest argument for the GPU/parallel crypto path—an argument that holds independently of the DEX and FHE results. These are the primitives from which the unified engine’s aggregate behaviour is composed; no single one is the engine.

15 Conclusion

Measured benchmarks demonstrate four independent acceleration vectors, from Apple M1 Max Metal to production NVIDIA GB10 and AMD RDNA silicon:

1. **GPU EVM kernel:** 85.9 M opcodes/sec, $4.85\times$ peak speedup over CPU. GPU wins above 500 opcodes/tx, reaching $3.09\times$ at 10,000 ops/tx.
2. **C++ interpreter:** 20.9 Ggas/s with full state ($5.1\times$ over Go EVM). 416 Ggas/s interpreter-only.
3. **Parallel ecrecover:** $7.1\times$ on 10 cores. This is the highest-leverage single optimization because ecrecover consumes 87% of block processing time.
4. **Production DEX matching:** on shipping accelerators, GPU-native batch matching exceeds **one billion orders/sec** on both an AMD Radeon 8060S (1.56 Gord/s, $2.7\times$ its 32-core CPU) and an NVIDIA GB10 (1.00 Gord/s, $1.8\times$ its Grace CPU), aggregating to 2.56 Gord/s across a two-node network.

GPU memory scales linearly at 73 KB/tx, supporting up to 800k transactions per block on 64 GB unified memory. The system is validated by 1,183 tests across C++, Rust, Go, Metal, and WGSL.

These optimizations are orthogonal and composable: parallel ecrecover + C++ EVM + GPU crypto + Block-STM can be combined for multiplicative throughput gains.

The unified engine, end to end. Composing the measured layers—billion-order GPU matching, a tuned 2.35 Gbit/s inter-node link at 94% of line rate, and post-quantum ML-DSA-65 at 134 μ s verification—a single colocated two-GPU venue over 800 Gbit/s InfiniBand admits an estimated 1.79 Gordons/s, exceeding the *entire world’s traded order flow* by more than two orders of magnitude while remaining fully post-quantum and, modulo an estimated $\sim 5\text{--}15\%$ confidential-compute overhead, fully confidential. Because the DEX matcher, the native GPU EVM (`block_stm.execute_kernel` and companions), and the 28-kernel FHE layer (NTT primitive at $23.6\times$ over CPU) share one unified-memory address space and a single dispatch stream, a confidential order is matched, settled, and homomorphically processed without ever leaving the accelerator. This is the defining property of the Lux unified GPU-native multichain engine: the empirical case that a post-quantum, confidential, GPU-native settlement fabric can clear all of the world’s markets from a single colocated site.

References

- [1] A. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, Y. Malkhi, Y. Xia, R. Zhou. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing*. PPOPP 2023.
- [2] Risechain. *pevm: Parallel EVM implementation in Rust*. <https://github.com/risechain/pevm>, 2024.
- [3] Paradigm. *reth: Modular, contributor-friendly and blazing-fast implementation of the Ethereum protocol, in Rust*. <https://github.com/paradigmxyz/reth>, 2023.
- [4] ipsilon. *evmone: Fast Ethereum Virtual Machine implementation*. <https://github.com/ipsilon/evmone>, 2019.
- [5] Hanzo AI. *ZAP: Zero-Copy Application Protocol*. <https://github.com/zap-proto/zap>, 2025.
- [6] Lux Industries, Inc. *Quasar: Post-Quantum Finality Consensus for Multi-Chain Networks*. Lux Technical Report, 2025.
- [7] Ethereum Foundation. *go-ethereum: Official Go implementation of the Ethereum protocol*. <https://github.com/ethereum/go-ethereum>, 2014.
- [8] Paradigm. *reth: Modular, contributor-friendly and blazing-fast implementation of the Ethereum protocol, in Rust*. <https://github.com/paradigmxyz/reth>, 2023.
- [9] Bortolomeazzi, S. and Konstantopoulos, G. *Reth’s path to 1 gigagas per second, and beyond*. Paradigm, April 2024. <https://www.paradigm.xyz/2024/04/reth-perf>
- [10] Erigon Tech. *erigon: Ethereum execution client (Go)*. <https://github.com/erigontech/erigon>
- [11] Chainstack. *Erigon vs Geth: Which Ethereum client is better*. <https://chainstack.com/ethereum-clients-geth-and-erigon/>, 2026.
- [12] ipsilon. *evmone: Fast Ethereum Virtual Machine implementation in C++*. <https://github.com/ipsilon/evmone>, 2019.

- [13] Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Malkhi, D., Xia, Y., Zhou, R. *Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing*. PPOPP 2023. arXiv:2203.06871.
- [14] Risechain. *pevm: Parallel EVM implementation in Rust*. <https://github.com/risechain/pevm>, 2024.
- [15] Bitcoin Core. *libsecp256k1: Optimized C library for ECDSA signatures on the secp256k1 curve*. <https://github.com/bitcoin-core/secp256k1>
- [16] Supranational. *blst: Multilingual BLS12-381 signature library*. <https://github.com/supranational/blst>
- [17] Knuth, D. E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. Algorithm D (multiple-precision division), §4.3.1. Addison-Wesley.
- [18] Lux Industries, Inc. *Lux Lightspeed DEX: On-Chain Order Book with Sub-Second Finality*. Lux Technical Report.